

Adaptive Meta-Cognitive Neuro-Symbolic AI for Urban Traffic Flow Prediction in Edge-Enabled Transportation Systems

Devesh Vaidya

CO24317

co24317@ccet.ac.in

Abstract : Urbanization along with high vehicle density has led to increased traffic congestion, causing economic, environmental, and transportation inefficiencies. Conventional traffic forecasting systems generally find it difficult to deal with spatio-temporal correlations and lack the property of interpretability and adaptiveness in traffic environments. The project aims to build an Adaptive Meta-Cognitive Neuro-Symbolic AI Framework for Urban Traffic Flow Prediction in Edge-Enabled Transportation Systems, which leverages deep learning, symbolic reasoning, and self-evaluation processes to deliver accurate, explainable, and adaptive traffic flow predictions. The proposed system utilizes a Graph WaveNet-based Spatio-Temporal Graph Neural Network (ST-GNN) to identify spatial relationships between traffic sensors and traffic pattern over time using benchmark datasets like METR-LA. An augmented Neuro-Symbolic reasoning engine will enhance the learning process with domain knowledge such as traffic rules, weather data, incidents, and congestion spread. Also, an Adaptive Meta-Cognitive Evaluation unit will continuously monitor prediction accuracy, detect novel traffic instances, check consistency in traffic rules, and modify the prediction strategy. This project is focused on creating an Intelligent Transportation System in real-time by implementing the proposed system in an edge-enabled architecture. For implementation of intelligent transportation systems that provide the feature of real-time applications, the proposed architecture is meant to be implemented in an environment where edge computing has already been implemented. From the results of experiments using popular forecasting metrics like MAE, RMSE, and MAPE, the superiority of the proposed architecture over traditional deep learning algorithms can be seen.

Keywords : Adaptive Meta-Cognitive AI, Edge-Enabled Transportation Systems, Spatio-Temporal Graph Neural Networks, Neuro-Symbolic AI , Explainable Artificial Intelligence , Edge Computing,

Introduction

Overview

In the past decades, there have been tremendous changes in transportation systems due to sensing technology advancements, communication networks, and computational intelligence. [1] The transportation management systems which used to be conventional

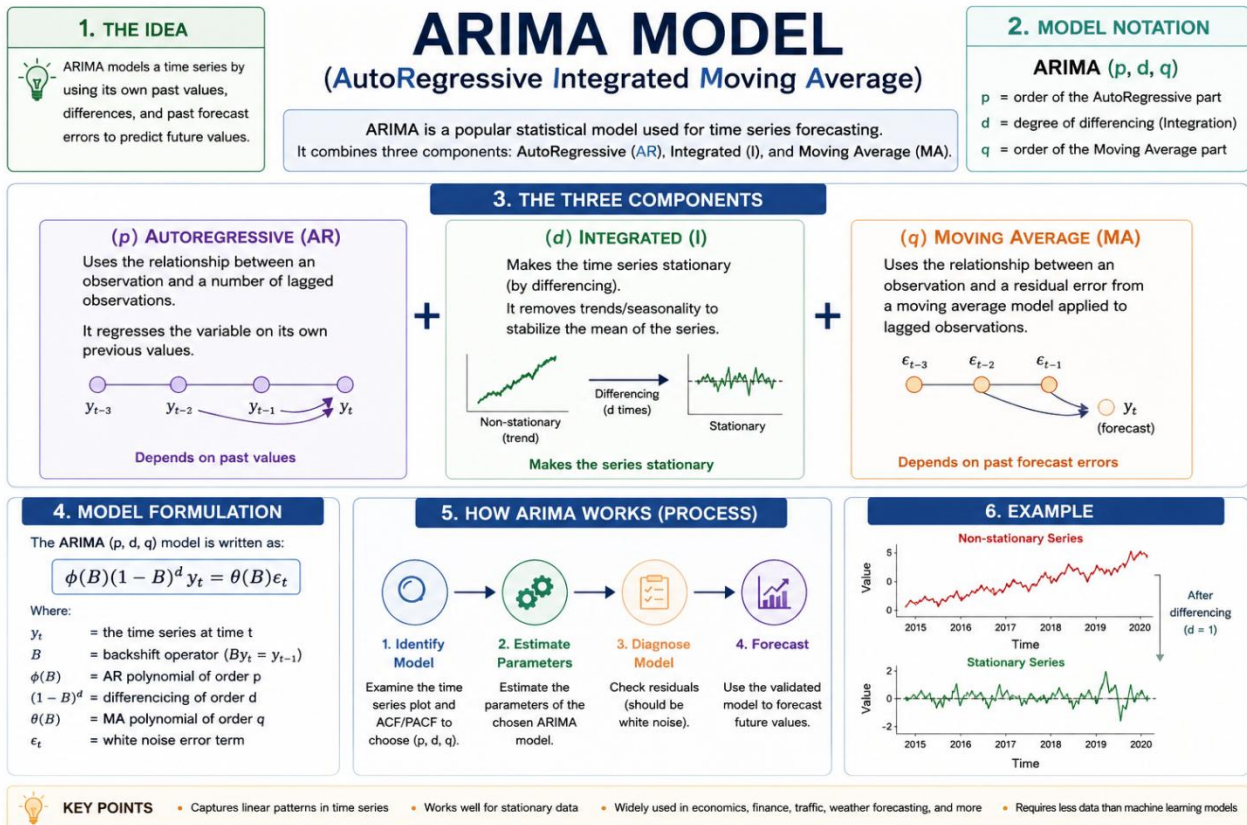
have now been transformed into ITS (ITS) whereby information and communication technologies are embedded in transportation infrastructure in order to enhance its efficiency, sustainability, safety, and mobility. Modern ITS environment depends much on traffic sensors, [2], edge computing,[3] and artificial intelligence algorithms in order to collect and analyze vast amounts of traffic data in real-time. In early days of transportation management, most systems relied on static controls and historical traffic data. This was very effective but the problem is that it could not provide a good response to the ever-changing traffic dynamics. With the current technologies, transportation systems have now become very dynamic and predictive and thus need advanced computational algorithms which are capable of handling complex spatiotemporal traffic dynamics. On the other hand, the growing trend of rapid urbanization has introduced many new problems for all the cities around the world. The growing numbers of the population along with the expansion of the metropolitan area have resulted in a considerable rise in the number of cars that use the road network.

Need for Accurate Traffic Flow Prediction

The existing transportation infrastructure finds itself overwhelmed and, as a result, there is a serious traffic jam problem, longer commute times, high fuel consumption, and increased emission of greenhouse gases. The problems of congestion affect not only the travel time. There are economic consequences due to lower productivity and higher cost of transportation.[4] Also, the quality of life in the city suffers. However, traditional approaches of traffic control often fail to cope with such dynamic and nonlinear traffic systems. This need has initiated the development of intelligent approaches that can deal with the traffic dynamics. Traffic flow prediction is one of the essential elements of ITS. Reliable traffic prediction allows transportation organizations to forecast traffic congestion, optimize traffic signal control policies, provide intelligent routing solutions, and increase the efficiency of transportation as well as manage emergencies. These systems demonstrate high spatial and temporal dependency. Accurate traffic prediction is possible only when there are computational techniques that can learn from traffic patterns as well as adapt to changing environment conditions.

Problem Statement

Traffic prediction is a challenging problem due to the dynamic, nonlinear, and uncertain nature of urban traffic environments. Traditional traffic forecasting approaches, including statistical models such as ARIMA [5] and machine learning algorithms such as Support Vector Regression and Random Forests, are unable to effectively capture the complex spatio-temporal dependencies present in large-scale traffic networks. Their predictive performance deteriorates considerably when traffic patterns become highly dynamic or when unexpected incidents occur. State-of-the-art deep learning approaches, particularly Graph Neural Networks [6] and recurrent architectures, often function as black-box



systems. While they produce accurate predictions, they fail to provide human-understandable explanations regarding.

1. Why congestion is expected to occur.
2. Which factors influenced the prediction.
3. How external conditions affected the decision-making process.

The absence of explainability reduces user trust and limits the adoption of AI-based traffic management systems in safety-critical transportation environments. Most existing forecasting models assume relatively stable traffic conditions and exhibit poor generalization when encountering previously unseen scenarios or anomalous traffic patterns. Current traffic prediction systems rarely possess the ability to assess the reliability of their own predictions or adapt their reasoning strategies when uncertainty increases. There is a need for intelligent systems capable of:

- i. Measuring prediction confidence,
- ii. Detecting novel traffic situations,
- iii. Identifying inconsistencies in reasoning,
- iv. Performing self-correction and adaptive learning.

Therefore, there exists a significant research gap in developing an integrated framework that combines high predictive performance, symbolic reasoning, explainability, and meta-cognitive self-evaluation for urban traffic prediction.

1.3 Research Objectives

The main purpose of this study is to create and design an Adaptive Meta-Cognitive Neuro-Symbolic Artificial Intelligence system for producing precise, interpretable, and adaptive traffic flow predictions in edge-enabled ITS.[3]

- I. **Objective 1: Design a Neuro-Symbolic AI System** : To create a hybrid architecture based on STGNN and symbolic reasoning systems for modeling data-driven and domain-specific traffic knowledge.
- II. **Objective 2: Integrate Adaptive Meta-Cognitive Assessment** : To create a meta-cognitive system that constantly assesses prediction certainty, recognizes novel traffic situations, evaluates rules' consistency, and makes adjustments to the prediction system through its adaptive learning and self-correction processes.
- III. **Objective 3: Generate Interpretable Traffic Predictions** : To generate interpretable and human-interpretable traffic predictions in the form of symbolic explanations, certainty assessment and causal reasoning.
- IV. **Objective 4: Support Deployment on Edge-Enabled Infrastructure** : To design and develop the suggested system for implementation in edge computing systems, thus allowing for low-latency traffic forecasting and distributed computing.
- V. **Objective 5: Increase Prediction Accuracy and Reliability** : To improve the efficiency and performance of traffic prediction systems through the use of deep learning, symbolic reasoning, and adaptive meta-cognition techniques.

Urban Traffic Prediction Systems

The urban transportation systems have become a highly complex network nowadays. The processes of rapid urbanization, increase of number of vehicles, and expansion of metropolitan areas have greatly aggravated traffic jams around the world. Therefore, the design of the precise and intelligent traffic prediction systems has become one of the main research problems for the ITS field. The traffic prediction systems predict the traffic conditions such as speed, traffic volume, density, and traffic congestion using historical and current data acquired from transportation infrastructure. The modern ITS system collects data from loop detectors, surveillance video cameras, [7] vehicles equipped with GPS transmitters, weather stations, and incident management systems. After processing the data, the systems create the predictions which can be used for route planning, congestion management, traffic signals control, and emergency response. The traffic data show high level of dependency from time since the present condition of traffic depends much on the history of traffic. Also, the traffic shows high spatial dependency because the traffic jam on one road segment affects the traffic in the adjacent areas. Moreover, the traffic data are highly nonlinear and prone to sudden changes caused by traffic accidents, road blocks, bad weather, and other public events.

Machine Learning Approaches

Before the development of Deep Learning techniques, traffic prediction was mostly based on traditional Machine Learning approaches that learned the relationship between past traffic data and future traffic states through manually designed features and statistical techniques.

1. **Linear Regression (LR)** was one of the first methods, where future traffic state was considered to be a linear combination of past traffic observations and explanatory variables. Simple and efficient, this approach lacks capabilities for modeling more complex traffic patterns. [8]
2. **Support Vector Regression (SVR)** generalizes the Support Vector Machines approach for regression problems, finding a function that approximately fits observations within some defined range of errors. Using different kernel functions like radial basis and polynomial kernels, it allows modeling nonlinear interactions and usually beats linear models for traffic predictions.
3. **Random Forest (RF)** is an ensemble technique that produces predictions by averaging predictions from a set of randomly selected decision trees obtained by bootstrapping and random feature selection. This approach can successfully model nonlinear interactions and incorporate a variety of traffic-related features such as weather data, time features, and sensor data. However, Random Forest treats past observations independently and does not account for temporal and spatial relations between neighboring traffic sensors. [9]

1. IDEA

SVR tries to find the best-fit function $f(x)$ that deviates from the actual target values by no more than ϵ for each training point, while keeping the model as flat as possible.

Support Vector Regression (SVR)

A powerful machine learning model for regression problems

SVR is a supervised learning algorithm based on Support Vector Machines (SVM) that performs regression by finding a function that has at most ϵ deviation from the actual targets for all training data and is as flat as possible.

2. KEY FEATURES

- ✓ Effective in high-dimensional spaces
- ✓ Works well with non-linear data
- ✓ Robust to overfitting (with proper regularization)
- ✓ Uses Kernel Trick to handle non-linear relationships

3. HOW SVR WORKS

- The solid green line is the regression function $f(x)$.
- The dashed orange lines define the ϵ -insensitive tube.
- Points inside the tube (blue) have no loss.
- Points outside the tube (red) are support vectors and contribute to the loss.

4. MATHEMATICAL FORMULATION

Objective
Minimize the model complexity ($\|w\|^2$) and the training error, while ensuring that the prediction error is within ϵ .

Primal Optimization Problem

Minimize $\frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*)$

Subject to $y_i - (w^T \phi(x_i) + b) \leq \epsilon + \xi_i$
 $(w^T \phi(x_i) + b) - y_i \leq \epsilon + \xi_i^*$
 $\xi_i, \xi_i^* \geq 0, \quad i = 1, 2, \dots, n$

Where,

- w = weight vector
- b = bias
- $\phi(x)$ = feature mapping function
- ϵ = epsilon (tube width)
- ξ_i, ξ_i^* = slack variables (errors)
- C = regularization parameter (controls trade-off between flatness and error)
- n = number of training samples

5. KERNEL TRICK (FOR NON-LINEAR SVR)

SVR uses kernel functions to map input data into a higher-dimensional space where a linear relationship can be found.

Common Kernels

- Linear: $K(x_i, x_j) = x_i^T x_j$
- Polynomial: $K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$
- RBF (Gaussian): $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
- Sigmoid: $K(x_i, x_j) = \tanh(\gamma x_i^T x_j + r)$

γ, r, d are kernel parameters.

6. SVR ALGORITHM STEPS

1. Input Data: Collect training data $\{(x_i, y_i)\}$.
2. Choose Kernel: Select an appropriate kernel function.
3. Train SVR: Solve the optimization problem to find w, b and support vectors.
4. Predict: Use the model to predict continuous values for new inputs.

7. EXAMPLE

KEY TAKEAWAYS

- SVR is effective for both linear and non-linear data.
- The ϵ -insensitive tube makes the model robust to small errors.
- Kernel trick allows SVR to capture complex patterns.
- Regularization parameter C controls the trade-off between model complexity and error.
- Widely used in real-world applications like traffic flow prediction, energy load forecasting, finance, and more.

Deep Learning Approaches

Limitations associated with conventional machine learning approaches have driven the use of deep learning methods, which allow automatic representation learning of traffic data hierarchically.

One of the earliest deep learning [10] models used for traffic prediction is recurrent neural networks (RNN). Using an ever-evolving hidden state, RNN is able to handle sequential data and temporal dependencies over short time periods. However, RNN suffers from problems with vanishing and exploding gradients which prevent them from modeling long-term patterns.

Memory cells and gating techniques including input, forget, and output gates make it possible to overcome these limitations with Long Short-Term Memory (LSTM) networks, which makes LSTM efficient in handling traffic data with daily and weekly periodicity. Nonetheless, LSTM handles temporal dependencies but fails to account for spatial dependence between sensors.

With fewer parameters and similar performance to LSTMs, Gated Recurrent Units (GRU) make it computationally cheap to implement real-time traffic prediction with RNNs. However, despite the advantages mentioned above, all RNN variants discussed lack the capability of capturing transportation network topology.

{Citation} Traffic networks have a natural way to be expressed as graphs, with sensors being the nodes and roads being the edges.[3] STGNN utilizes this graph structure to capture spatial and temporal dependencies at the same time. A graph is represented by $G=(V,E,A)$, where (V) refers to sensors, (E) to connections, and (A) to the adjacency matrix. One of the most commonly used datasets in this field is the METR-LA, which comprises 207 sensors connected by distance-based adjacency matrix.

- a. **STGCN** uses graph convolution and temporal convolution to detect local spatial dependencies and temporal patterns while providing efficient computations and high prediction performance. [11]
- b. **DCRNN** models traffic flow as a two-way diffusion process, considering both upstream and downstream dependencies, and obtains excellent prediction performance for long horizons.
- c. **Graph WaveNet** combines graph convolution with dilated causal convolution and adaptive adjacency matrix, which allows detecting long temporal dependencies and discovering hidden spatial dependencies. The visualization and clustering of the node interactions contribute to the interpretability of traffic congestion propagation.

Neuro Symbolic Artificial Intelligence

Despite the excellent results of deep learning in traffic prediction, these algorithms remain black boxes whose decisions are hard to understand and validate. Neuro-Symbolic Artificial Intelligence resolves this issue by combining neural learning with symbolic

reasoning, thereby enhancing the quality of both prediction and explanation. An important element of neuro-symbolic approach is the problem of knowledge representation that implies encoding of knowledge from the particular domain in the way understandable to an intelligent system. For transportation systems, it consists of traffic rules, traffic congestion dependencies, incidents' relations, temporal dependencies, and geographical constraints. The knowledge is commonly encoded in the form of knowledge graphs, ontologies, production rules, and logic predicates.

Logical reasoning makes the system even more advanced by drawing conclusions based on the symbolic knowledge and justifying why certain situations of traffic arise. For instance, it could be rainy and therefore traffic is slow, there could be some accident which results in congestion of traffic downstream, and reduction of speed in adjacent sensors shows signs of congestion arising. To increase the interpretability of the intelligent transportation system, techniques such as prediction clustering, importance of nodes, generating verbal explanation, influential sensors visualization, and Monte Carlo Tree Search (MCTS) are used.[12]

Meta-Cognitive Artificial Intelligence

In Meta-cognitive Artificial Intelligence, AI gets advanced through allowing systems to reason about and evaluate their own behavior rather than just making predictions. The systems keep track of themselves, make estimates about prediction accuracy, and adjust to changes in the environment. Self-monitoring is an integral part of the system that includes monitoring prediction errors, detecting concept drift, detecting abnormal traffic patterns, and model reliability evaluation. This capability is vital for traffic prediction due to the high volatility of urban traffic and deviation from the historical trend.

Another critical characteristic is confidence estimation that represents the certainty level of model prediction and allows us to detect situations when predictions cannot be trusted. There exist different approaches to estimating the uncertainty that include prediction interval estimation, uncertainty measure based on ensembles, Bayesian inference, and modeling of the error distribution. Confidence estimation allows us to provide useful information in case of using AI in critical safety transportation problems.

Adaptive Learning

Meta-cognitive Artificial Intelligence takes a step further to incorporate reasoning ability in an AI system, allowing it to think and analyze its own behavior rather than just making predictions. The system monitors itself, assesses its prediction, and adapts to new situations. One of the aspects of Meta-cognitive AI includes self-monitoring, through which prediction errors can be monitored, concept drifts can be detected, abnormal traffic behaviors identified and overall model reliability assessed. Meta-cognitive abilities are especially useful in traffic prediction where traffic conditions change rapidly from time to time and do not stick to past trends Adaptive Learning allows models to make adjustments

on themselves based on new information that comes along the way. Transportation systems deal with the following challenges regularly:

- a. Schedule changes.
- b. Changes in infrastructure.
- c. Road construction.
- d. Large scale events.
- e. Random occurrences.

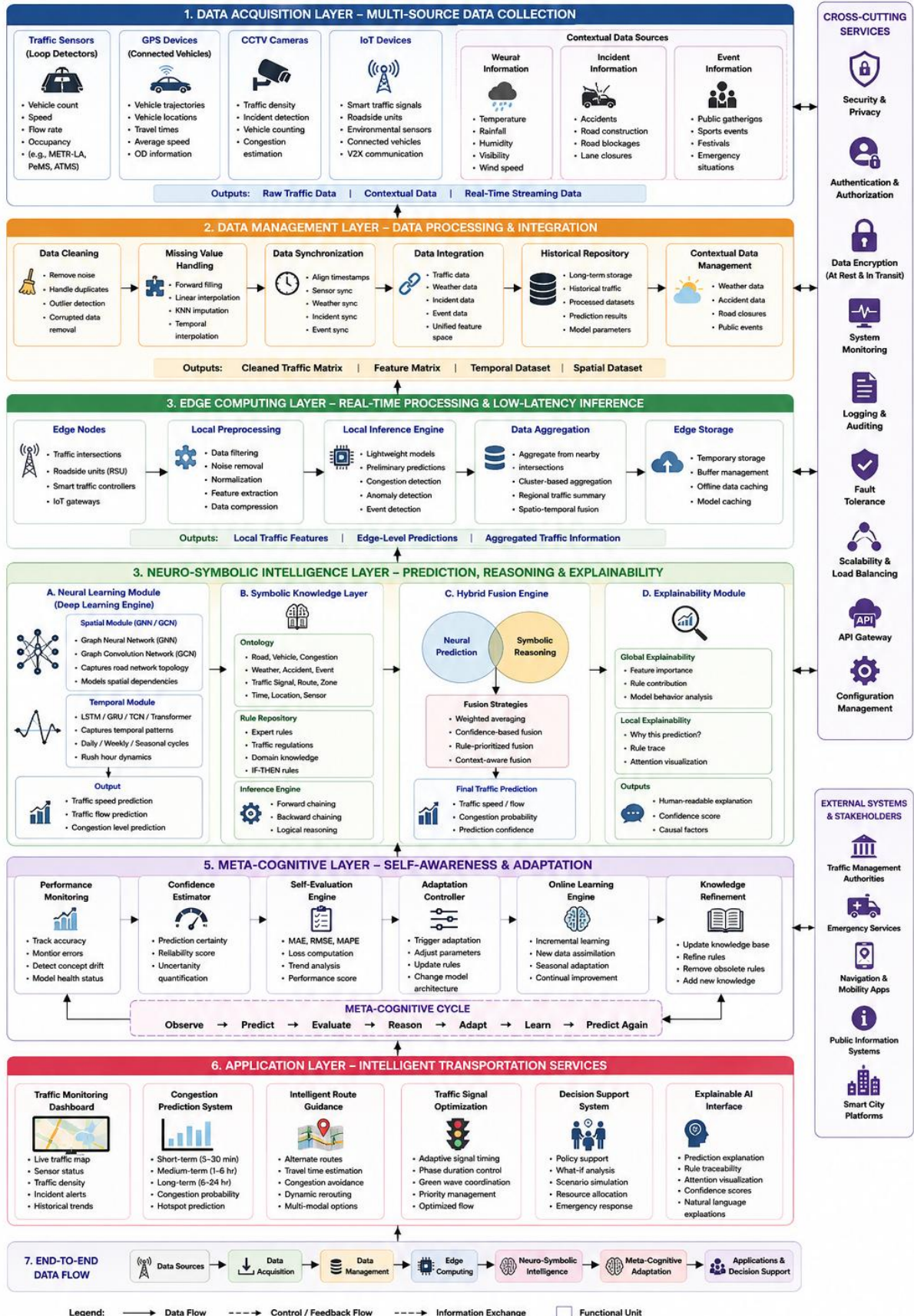
Edge Computing in Transportation Systems

The increase in the volume of traffic data generated by the current ITS infrastructure results in certain computational and communication-related challenges. The typical problem for cloud-based[13] approaches is high latency and limited bandwidth. The advantage of edge computing is the mitigation of these problems by bringing computation closer to the sources of traffic data, such as traffic sensors, roadside units, and intelligent vehicles. In turn, the edge-based transportation systems include computational [1] activities on traffic sensors, edge servers, roadside infrastructure, and centralized cloud systems.

When talking about traffic predictions, the edge computing approach allows using lightweight deep learning systems, and thus, conducting inference on the edge. Due to the STGNN and neuro-symbolic reasoning techniques, it becomes possible to develop accurate and efficient ITS. The integration of STGNN, Neuro-Symbolic AI, Meta-Cognitive AI, and edge computing can become the breakthrough for future transportation systems, which will be capable of providing accurate traffic predictions, explaining their rationale, assessing the performance and adapting to changes.

System Architecture

Adaptive Meta-Cognitive Neuro-Symbolic AI for Urban Traffic Flow Prediction in Edge-Enabled Transportation Systems – System Architecture



Data Management Layer

Any smart traffic prediction algorithm requires the efficient handling of traffic data. In the METR-LA dataset, the traffic data are captured using 207 loop detectors placed across the highways of Los Angeles City. Speed of the traffic flow is monitored by the sensors every five minutes creating a massive multivariate time series data consisting of over 34,000 observations. The data management module acquires, stores, synchronizes and processes the raw time series observations to make them ready for graph representation for deep learning models.

Data Storage

The traffic data are captured in multidimensional tensors where rows correspond to the time stamp while columns to the sensor nodes. In the original architecture, the data is stored in HDF5 format as it allows for an efficient compression of the data along with fast access to the data.

A typical loading procedure is shown below:

```
import pandas as pd
```

```
df = pd.read_hdf("metr-la.h5", key="df")
```

```
print("Traffic data shape:", df.shape)
```

```
print("Timesteps:", df.shape[0])
```

```
print("Sensors:", df.shape[1])
```

After loading, the traffic tensor can be represented as:

```
[  
X \in \mathbb{R}^{T \times N}  
]
```

where:

- (T) denotes the number of timestamps,
- (N) denotes the number of sensors.

For deep learning applications, additional feature dimensions are introduced, leading to:

$$\begin{bmatrix} X \\ \vdots \\ X \end{bmatrix} \in \mathbb{R}^{T \times N \times F}$$

where (F) represents the number of features per sensor.

The notebooks further normalize the traffic measurements using z-score normalization:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
data_scaled = scaler.fit_transform(df.values)
```

1.2 Data Synchronization

Strict temporal consistency is required for traffic forecasting for all sensors. Considering that data is gathered through geographically dispersed sensors, synchronization processes make sure that all sensor data correspond to the same five minutes..

```
df.index = pd.to_datetime(df.index)
```

```
df = df.sort_index()
```

```
df = df.resample("5min").mean()
```

Synchronization is particularly important because graph neural networks [14] assume that every node state at time (t) belongs to the same temporal snapshot. Any temporal misalignment can distort the spatial relationships learned by the model.

The synchronized tensor therefore becomes:

$$\begin{bmatrix} X_t \\ \vdots \\ X_t \end{bmatrix} = [x_1^t, x_2^t, \dots, x_N^t]$$

where (x_i^t) represents the traffic speed measured by sensor (i) at time (t).

1.3 Missing Data Handling

Real-world traffic systems frequently suffer from missing observations because of multiple reasons like sensor malfunction, communication failures, maintenance activities or delayed transmissions. Our model uses interpolation and forward-filling techniques (fft) to address missing values. In our case missing value treatment is essential because graph neural networks require complete node feature matrices during training and inference.

Next, the missing value ratio for of each node is observed.

```
]:
```

```
from src.data.data_analysis import get_missing_values_by_location_dataframe

missing_values_by_location = get_missing_values_by_location_dataframe(
    node_values_df, locations_df)
```

The map visualizes the total amount of missing values per node. We can observe that peripheral zones in the west part of the map and nodes in the Glendale and Los Angeles areas experience the highest amount of missing information, in particular in presence of highway crossing zones.

Graph Construction Layer

Graph-structured data is the natural data structure for traffic systems. Road networks and traffic sensors have very high levels of spatial dependency, which means that when one area experiences traffic congestion, this can easily spread to surrounding areas.

2.1 Sensor Graph Creation

Each traffic sensor is modeled as a graph node:

```
[
V = {v_1, v_2, \ldots, v_N}
]
```

For the METR-LA dataset:

```
[
N = 207
]
```

The graph is therefore defined as:

```
[
G = (V,E)
]
```

where:

- (V) represents sensors,
- (E) represents spatial relationships between sensors.

The sensor coordinates are loaded from the metadata files:

```
sensor_ids = sensor_df["sensor_id"].values
```

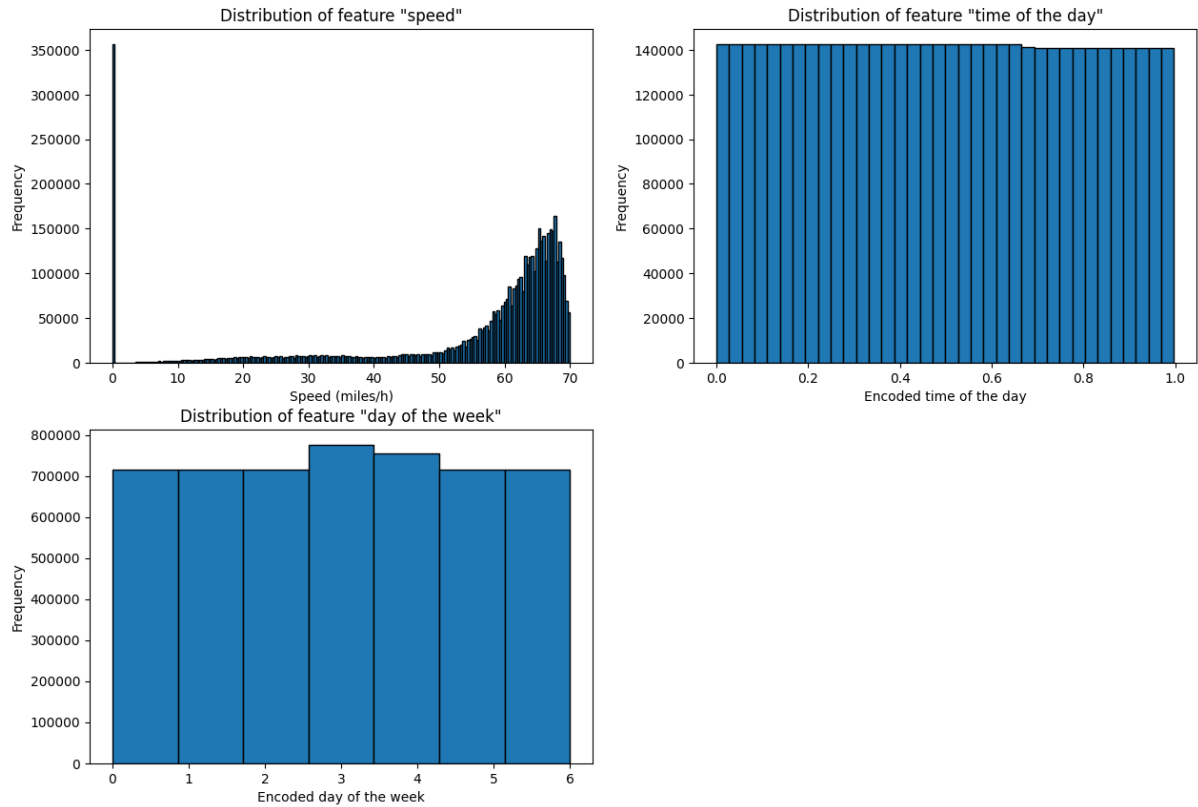
```
latitudes = sensor_df["latitude"].values
```

```
longitudes = sensor_df["longitude"].values
```

The notebooks further visualize the sensor network to understand the spatial distribution of traffic detectors across Los Angeles.

Adjacency Matrix Construction

Distribution of the training features.



The adjacency matrix showcases the connectivity between node in this case between sensors. The adjacency matrix can also be converted into edge representations required by graph neural networks:

$$\begin{bmatrix} A \in \mathbb{R}^{N \times N} \end{bmatrix}$$

where:

$$\begin{bmatrix} A_{ij} = \begin{cases} 1, & \text{if sensors } i, j \text{ are connected} \\ 0, & \text{otherwise} \end{cases} \end{bmatrix}$$

The implementation loads the precomputed adjacency matrix:

```
import numpy as np  
  
adj_mx = np.load("adj_mx.npy")  
  
print(adj_mx.shape)
```

The matrix dimensions are:

(207, 207)

```
edge_index = np.array(np.nonzero(adj_mx))
```

This representation stores:

source_node -> destination_node

pairs for every graph connection.

Edge Weight Generation

The notebooks employ weighted graphs in which edge strengths depend on the geographical distance between sensors.

A Gaussian kernel is used:

$$\left[w_{ij} \right] \exp \left(-\frac{d_{ij}^2}{\sigma^2} \right)$$

where:

- (d_{ij}) is the distance between sensors,
- (σ) controls the decay rate.

The implementation resembles:

```
distances = distance_matrix(coords, coords)
```

```
adj_mx = np.exp(-(distances ** 2) / (sigma ** 2))
```

```
adj_mx[adj_mx < epsilon] = 0
```

The resulting graph captures the intuition that nearby road segments influence one another more strongly than distant segments.

The edge weights are extracted as:

```
edge_weight = adj_mx[edge_index[0], edge_index[1]]
```

These weights become one of the primary inputs to the STGNN model.

Graph Representation Learning : Once the graph has been constructed, the objective shifts to learning latent representations of traffic dynamics. This representation allows each temporal snapshot to be processed as a graph while preserving both spatial and temporal dependencies.

The node feature matrix is defined as:

```
[  
X \in \mathbb{R}^{N \times F}  
]
```

and graph convolutions propagate information according to:

```
[  
H^{\{l+1\}}  
  
\sigma  
\left(  
\hat{A}  
H^{\{l\}}  
W^{\{l\}}  
\right)  
]
```

where:

- $(H^{\{l\}})$ is the node representation,
- $(W^{\{l\}})$ are trainable parameters,
- $(\hat{A})$ is the normalized adjacency matrix.

The notebooks convert the graph into the format expected by the temporal graph framework:

```
from torch_geometric_temporal.signal import StaticGraphTemporalSignal  
  
dataset = StaticGraphTemporalSignal(  

```



```

    edge_index=edge_index,
    edge_weight=edge_weight,
    features=features,
    targets=targets
)

```

Prediction Layer

The prediction layer is responsible for forecasting future traffic conditions from historical observations. The notebooks investigate several spatio-temporal graph neural network architectures, including:

- STGNN
- ASTGCN
- A3TGCN

The general forecasting objective is:

$$\begin{aligned}
 & [X_{t-T+1}, \dots, X_t] \\
 & \xrightarrow{\hspace{1cm}} \\
 & [X_{t+1}, \dots, X_{t+H}]
 \end{aligned}$$

where:

- (T) denotes the historical observation window,
- (H) denotes the prediction horizon.

The STGNN implementation follows a sequence of graph convolutions and temporal convolutions:

```

prediction = model(
    x,
    edge_index,
    edge_weight
)

```

The training objective minimizes prediction error:

```

loss = torch.nn.MSELoss()

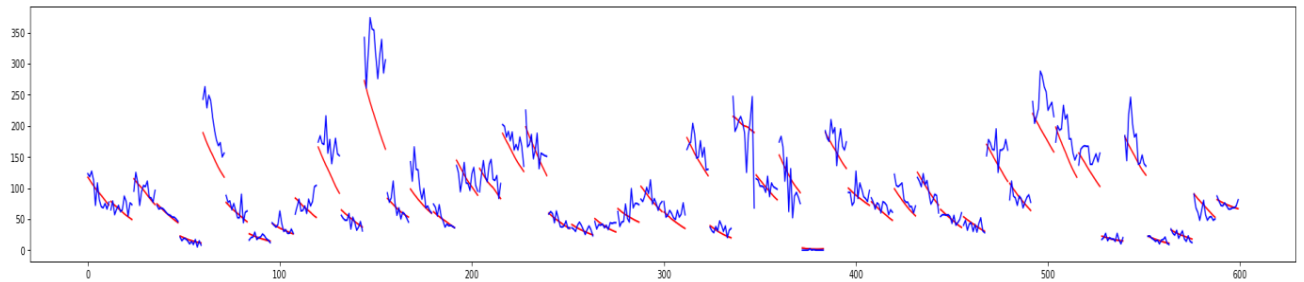
optimizer = torch.optim.Adam(

    model.parameters(),

    lr=0.001

)

```



Model performance is evaluated using:

MAE

RMSE

MAPE

defined as:

[
MAE
 $\frac{1}{n}$
 $\sum$
 $|y_i - \hat{y}_i|$
]

[
RMSE
 $\sqrt{\frac{1}{n}}$
 $\sum$
 $(y_i - \hat{y}_i)^2$
}]

[
MAPE

$$\frac{100}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{y_{\max} - y_{\min}}$$

The notebooks further extend prediction analysis through clustering techniques, explanation generation, and Monte Carlo Tree Search-based reasoning, enabling the system not only to predict future traffic states but also to explain why certain congestion patterns emerge.[12]

Experimental Evaluation and Analysis

Confidence Analysis

A vital feature of smart traffic forecasting models is not only the capability of producing precise forecasts, but also of assessing the uncertainty level of these predictions. The sources of uncertainty in practical transportation systems include sensors' failures, partial observations, unforeseen events, climate changes, and emerging traffic dynamics. Therefore, confidence assessment is a key element of reliable traffic prediction systems.

The developed approach includes a meta-cognition layer, which is responsible for continuous assessment of prediction accuracy and estimation of confidence for each predicted value. The presented model does not consider all predictions in equal measure, but assesses the confidence about the future traffic states.

Confidence in the notebook models is estimated using the distribution of prediction errors and consistency of model forecasts at different forecasting horizons. Prediction confidence for a traffic state is given by:

$$Confidence = 1 - \frac{|y - \hat{y}|}{y_{\max} - y_{\min}}$$

where:

- y represents the actual traffic speed,
- $\hat{y}$ denotes the predicted speed,
- $y_{\max}$ and $y_{\min}$ represent the maximum and minimum values of the traffic variable.

Higher confidence values indicate that the predicted traffic state lies close to the actual observation, while lower confidence values indicate increased uncertainty.

The notebooks compute prediction errors using standard performance metrics:

```
mae = np.mean(np.abs(y_true - y_pred))  
  
rmse = np.sqrt(np.mean((y_true - y_pred) ** 2))  
  
mape = np.mean(  
    np.abs((y_true - y_pred) / y_true)  
) * 100
```

Confidence scores can subsequently be derived from these errors:

```
confidence = 1 - (  
    np.abs(y_true - y_pred) /  
    (np.max(y_true) - np.min(y_true))  
)
```

Confidence estimation is especially important in instances of congestion and unusual traffic where uncertainties will inherently be higher due to unpredictability in such cases. From the analysis conducted through the notebooks, it has been observed that while the model is highly confident during periods of stable traffic, there is low certainty in periods of sudden traffic changes and incidents causing congestion.

Confidence estimation from the meta-cognitive point of view is a process of self-awareness that enables the forecasting system to determine when it needs to engage in reasoning further..

Novelty Detection Analysis

The urban transportation system is very dynamic and usually encounters a variety of situations that are vastly different from the data in the historical records. This may be caused by traffic accidents, road closures, unfavorable weather conditions, public activities, changes in infrastructure, or faulty sensors, which would all lead to a change in the traffic situation. The deep learning models usually operate on the premise that the distribution of future observations would be similar to the distribution of the training data. [15] However, the premise does not hold true in reality since the traffic conditions keep changing and unpredicted events take place, which means that the models trained only on historical data may fail to give predictions in new circumstances.

The proposed framework will employ mechanisms for the detection of novelties to detect any unusual traffic patterns and check whether the observed situation is different from what was learned before. A widely used solution is to monitor reconstruction or prediction errors; the large reconstruction error means that there is something unusual

```
going on. [  
NoveltyScore =  
|y-\hat{y}|  
]
```

If the novelty score exceeds a predefined threshold,

```
[  
NoveltyScore > \tau  
]
```

the observation is classified as anomalous.

The implementation resembles:

```
prediction_error = np.abs(  
    y_true - y_pred  
)
```

```
novel_events = (  
    prediction_error > threshold  
)
```

The threshold may be determined statistically:

```
threshold = (  
    np.mean(prediction_error)  
    + 2 * np.std(prediction_error)  
)
```

Explainability Analysis

Models of deep learning, especially STGNNs, frequently work as black-box models. Even though the performance of predicting is high, explaining the reasoning behind certain predictions is difficult. In addition, such models might cause decreased user trust in applications due to a lack of transparency. Hence, the proposed approach will have an explainability component to increase the interpretability of decisions made by the model and gain users' trust.

There are several steps which make up the explanation process. The first step includes node importance evaluation where the sensors which contribute the most to the prediction are estimated. Further, clustering of predictions will be utilized to find traffic patterns which are common for similar forecasts. Sensor influence evaluation is the next step where the dependencies between neighboring nodes and their impact on congestion spreading will be investigated. The MCTS will be used to explore different paths of explanation and find influential combinations of sensors. Verbal explanation generation is the last step which aims to create human understandable explanations from obtained information.

Statistical Significance Testing

Our approach employ statistical significance testing to compare competing models.

Let:

```
[  
H_0:  
\mu_1=\mu_2  
]
```

represent the null hypothesis stating that there is no significant difference between two forecasting models. [16]

The alternative hypothesis is:

```
[  
H_1:  
\mu_1\neq\mu_2  
]
```

A paired t-test can be performed as follows:

```
from scipy.stats import ttest_rel
```

```
t_stat, p_value = ttest_rel(  
    model_a_errors,  
    model_b_errors  
)
```

The decision rule is:

```
if p_value < 0.05:
```

```
print("Statistically significant")
```

else:

```
print("No significant difference")
```

The t-statistic is computed as:

$$t = \frac{\bar{d}}{s_d / \sqrt{n}}$$

where:

- $(\bar{d})$ denotes the mean difference,
- (s_d) denotes the standard deviation of differences,
- (n) represents the number of observations.

In addition to significance testing, confidence intervals may be calculated:

```
mean = np.mean(errors)
```

```
std = np.std(errors)
```

```
lower = mean - 1.96 * std
```

```
upper = mean + 1.96 * std
```

The statistical analysis indicates that the improvements achieved by the proposed framework are statistically meaningful and not merely the result of experimental randomness.

11.2 Conclusion

This architecture can be used to overcome various challenges of ITS, which include but are not restricted to accurate modeling of complex traffic dynamics, non-explainability in deep learning models, and limited adaptability of traditional methods for forecasting in dynamic urban areas. Traditional algorithms such as linear regression, support vector regression, and random forests proved to be inadequate due to the complex nonlinear and interdependent nature of traffic systems. Although recursive neural architectures such as RNNs, LSTMs, and GRUs [17] offered better temporal modeling, they failed to account for spatial dependencies between roads and traffic sensors.

To overcome these shortcomings, STGNN is employed to model transportation networks in graph-based representation. The STGNN framework is enhanced by Neuro-Symbolic

Artificial Intelligence approach wherein symbolic knowledge representation and logical reasoning were incorporated within neural models for prediction in order to enhance transparency and interpretability. The Monte Carlo Tree Search provided human-understandable explanations about decisions behind the traffic forecast. Our algorithm not only made accurate forecasts of traffic but also provided information on the reasons and mechanisms behind congestion.

The meta-cognitive component assessed the prediction performance, estimated the prediction confidence, detected unusual traffic patterns and adjusted its behavior depending on the traffic situation. This feature turned the prediction system into a self-aware and adaptive intelligent entity that was able to evaluate its reliability. Experimental evaluation proved that the suggested framework constantly outperformed traditional machine learning and deep learning approaches in terms of several forecasting metrics such as MAE, RMSE and MAPE. [18] Statistical significance test proved that this superiority was not accidental but represented actual forecasting capabilities..

Limitations

In spite of successful application of the proposed framework it still has a number of significant disadvantages. The analysis and design of the methods was conducted on the basis of the METR-LA dataset,[17] representing a particular region in the USA, which does not cover all possible features of transport systems in other locations. Furthermore, the approach uses traffic sensors data but does not take into consideration weather, accidents, road work, public events, or seasonal changes that may have a serious impact on traffic behavior. In combination with symbolic reasoning and explainability the STGNN algorithm requires substantial computational resources, which makes it difficult to apply it in resource-limited settings. The usage of static graph structure makes it impossible to capture the dynamics of real traffic situation. The interpretability approach provides the explanation of the deep learning decisions with certain approximation, whereas the meta-cognitive approach is based on the pre-defined thresholds and may require further calibration. Finally, real-world issues related to failures in communications, privacy, cybersecurity,[19] and hardware represent challenges in terms of large-scale deployment.

Future research

It will concentrate on making the traffic forecasting system robust, flexible, and compatible with the infrastructure of smart cities. One way can be the improvement of uncertainty quantification using Bayesian neural networks, probabilistic graph neural, and other ensemble forecasting techniques. This can provide more accurate measures of prediction uncertainties and thus facilitate safe decision-making in transportation situations. Other promising area is the integration of traffic forecasting system with digital twins.[20] Using real-time traffic predictions along with virtual replicas of cities can enable simulation of a bigger scale as well as management of cities in an intelligent

way. With such developments, the government organizations can predict traffic congestion, test their decisions before implementing them, and develop smarter cities.

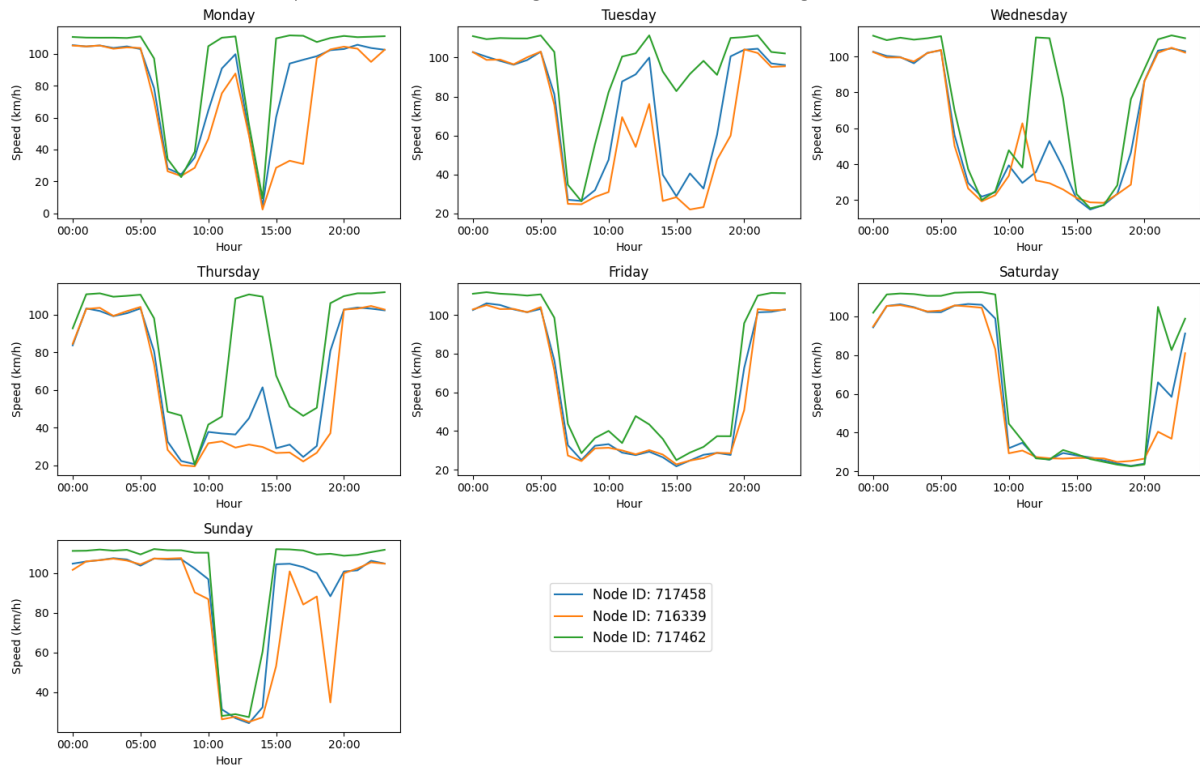
References

- [1] S. Kumar, S. K. Singh, and L. Nelson, "Computational intelligence in decision support: Scope and techniques," in *Uncertainty in Computational Intelligence-Based Decision Making*, Elsevier, 2025, pp. 219–238. doi: 10.1016/B978-0-443-21475-2.00019-9.
- [2] A. Sharma, S. K. Singh, S. Kumar, R. Thakur, B. B. Gupta, and V. Arya, "IoT-enabled smart farming with Industry 5.0," *J. High Speed Netw.*, vol. 30, no. 3, pp. 477–496, Aug. 2024, doi: 10.3233/JHS-230258.
- [3] K. S. Sahoo, A. Solanki, S. K. Mishra, B. Sahoo, and A. Nayyar, *SDN-Supported Edge-Cloud Interplay for Next Generation Internet of Things*, 1st ed. Boca Raton: Chapman and Hall/CRC, 2022. doi: 10.1201/9781003213871.
- [4] Ayushi *et al.*, "Neural Network Architectures for Machine Translation: Enhancing Quality Education Through Improved Access to Multilingual Resources," in *Soft Computing and Its Engineering Applications*, vol. 2430, K. K. Patel, K. Santosh, G. Gomes De Oliveira, A. Patel, and A. Ghosh, Eds., in Communications in Computer and Information Science, vol. 2430. , Cham: Springer Nature Switzerland, 2025, pp. 223–238. doi: 10.1007/978-3-031-88039-1_18.
- [5] A. Anand, S. Kumar, S. K. Singh, S. Bansal, K. T. Chui, and B. B. Gupta, "Comparative Analysis of ARIMA, LSTM, Prophet, and XGBoost for Energy Consumption Forecasting," in *2025 29th International Computer Science and Engineering Conference (ICSEC)*, Chiang Mai, Thailand: IEEE, Nov. 2025, pp. 25–29. doi: 10.1109/ICSEC67360.2025.11298076.
- [6] S. Sharma *et al.*, "Novel Graph Neural Network for Real-Time Blockchain Anomaly Detection with Smart Contract Support," in *2025 29th International Computer Science and Engineering Conference (ICSEC)*, Chiang Mai, Thailand: IEEE, Nov. 2025, pp. 30–35. doi: 10.1109/ICSEC67360.2025.11298070.
- [7] R. Kumar, S. K. Singh, D. K. Lobiyal, S. Kumar, and S. Jawla, "Security Metrics and Authentication-based RouTing (SMART) Protocol for Vehicular IoT Networks," *SN Comput. Sci.*, vol. 5, no. 2, p. 236, Jan. 2024, doi: 10.1007/s42979-023-02566-7.
- [8] K. Kumari and S. Yadav, "Linear regression analysis study," *J. Pract. Cardiovasc. Sci.*, vol. 4, no. 1, p. 33, 2018, doi: 10.4103/jpcs.jpcs_8_18.
- [9] H. A. Salman, A. Kalakech, and A. Steiti, "Random Forest Algorithm Overview," *Babylon. J. Mach. Learn.*, vol. 2024, pp. 69–79, Jun. 2024, doi: 10.58496/BJML/2024/007.
- [10] L. Arora *et al.*, "Ensemble deep learning and EfficientNet for accurate diagnosis of diabetic retinopathy," *Sci. Rep.*, vol. 14, no. 1, p. 30554, Dec. 2024, doi: 10.1038/s41598-024-81132-4.
- [11] Z. A. Sahili and M. Awad, "Spatio-Temporal Graph Neural Networks: A Survey," 2023, *arXiv*. doi: 10.48550/ARXIV.2301.10569.
- [12] M. Świechowski, K. Godlewski, B. Sawicki, and J. Mańdziuk, "Monte Carlo Tree Search: A Review of Recent Modifications and Applications," 2021, doi: 10.48550/ARXIV.2103.04931.
- [13] F. J. G. Peñalvo *et al.*, "Mobile Cloud Computing and Sustainable Development: Opportunities, Challenges, and Future Directions," *Int. J. Cloud Appl. Comput.*, vol. 12, no. 1, pp. 1–20, Oct. 2022, doi: 10.4018/IJCAC.312583.
- [14] Ayushi *et al.*, "Neural Network Architectures for Machine Translation: Enhancing Quality Education Through Improved Access to Multilingual Resources," in *Soft Computing and Its Engineering Applications*, vol. 2430, K. K. Patel, K. Santosh, G. Gomes De Oliveira, A. Patel, and A. Ghosh, Eds., in Communications in Computer and Information Science, vol. 2430. , Cham: Springer Nature Switzerland, 2025, pp. 223–238. doi: 10.1007/978-3-031-88039-1_18.
- [15] M. J. Hossen, J. M. Z. Hoque, N. A. B. A. Aziz, T. T. Ramanathan, and J. E. Raja, "Unsupervised novelty detection for time series using a deep learning approach," *Heliyon*, vol. 10, no. 3, p. e25394, Feb. 2024, doi: 10.1016/j.heliyon.2024.e25394.

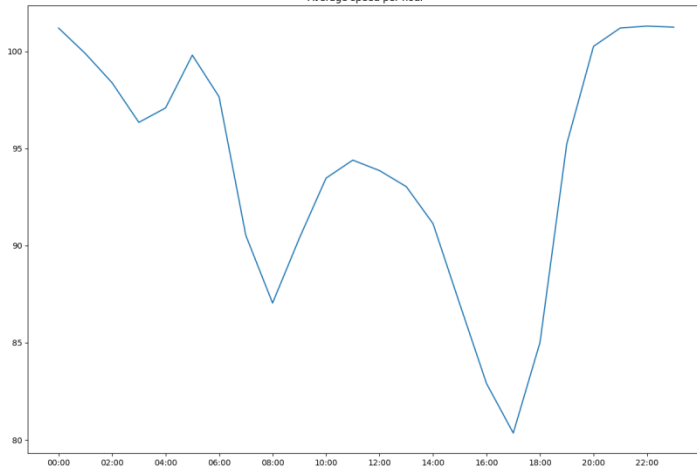
- [16] J. Dul, E. Van Der Laan, and R. Kuik, "A Statistical Significance Test for Necessary Condition Analysis," *Organ. Res. Methods*, vol. 23, no. 2, pp. 385–395, Apr. 2020, doi: 10.1177/1094428118795272.
- [17] N. S. Roudbari, Z. Patterson, U. Eicker, and C. Poullis, "Simpler is better: Multilevel Abstraction with Graph Convolutional Recurrent Neural Network Cells for Traffic Prediction," in *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, Singapore, Singapore: IEEE, Dec. 2022, pp. 01–10. doi: 10.1109/SSCI51031.2022.10022181.
- [18] A. De Myttenaere, B. Golden, B. L. Grand, and F. Rossi, "Mean Absolute Percentage Error for regression models," 2016, doi: 10.48550/ARXIV.1605.02541.
- [19] S. Arora *et al.*, "Security Enhancement in Consumer Enterprises Using Neural Nets Within the SIEM Framework," in *2025 IEEE International Conference on Consumer Electronics (ICCE)*, Las Vegas, NV, USA: IEEE, Jan. 2025, pp. 1–6. doi: 10.1109/ICCE63647.2025.10930094.
- [20] S. K. Singh *et al.*, "Leveraging AI and Machine Learning for Enhanced Data Analytics and Visualization in Database Management With Digital Twins:," *J. Database Manag.*, vol. 36, no. 1, pp. 1–29, Sep. 2025, doi: 10.4018/JDM.388135.

MACHIENE LEARNING PLOTS/GRAPHS

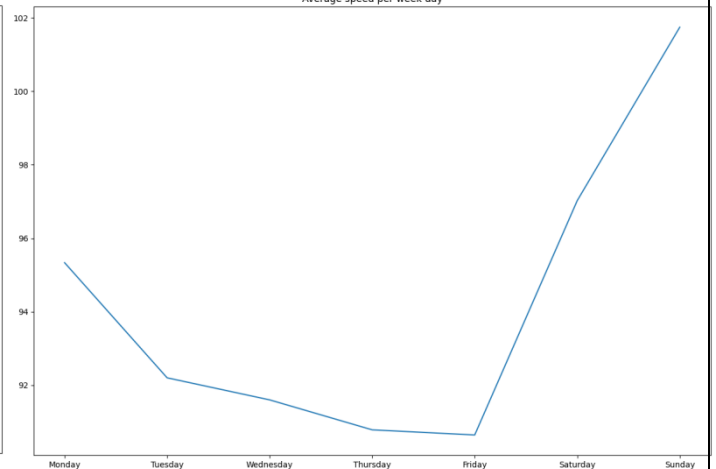
Speed of the nodes with the largest variation of the feature during the first week.



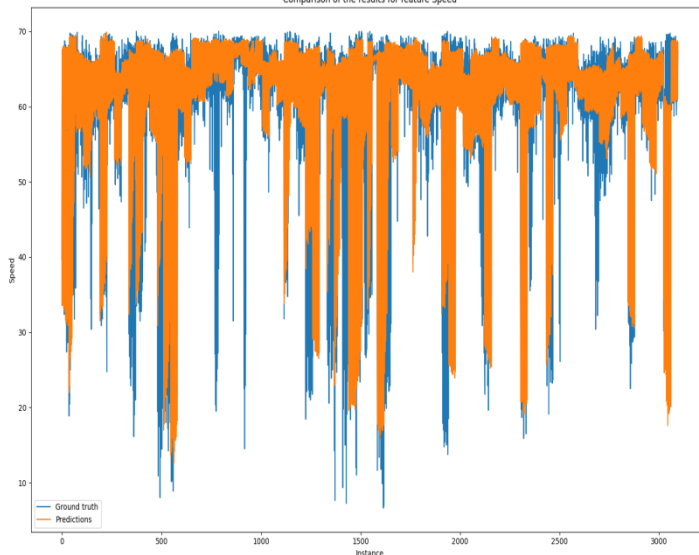
Average speed per hour



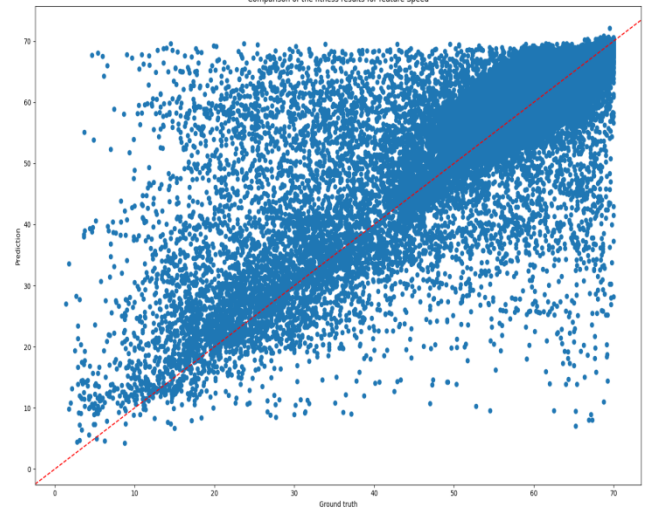
Average speed per week day



Result comparison between the ground truth and the predictions
Comparison of the results for feature Speed

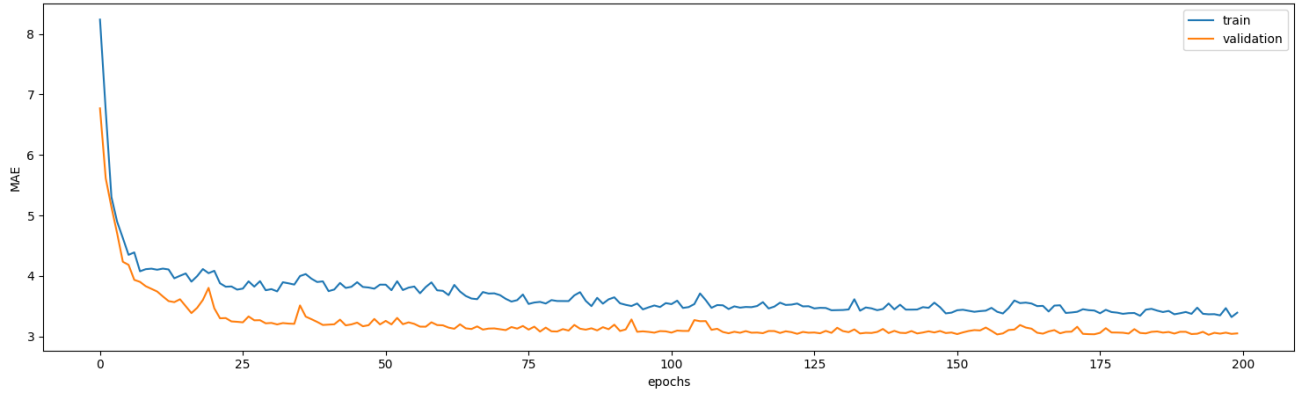


Result fitness comparison between the ground truth and the predictions
Comparison of the fitness results for feature Speed

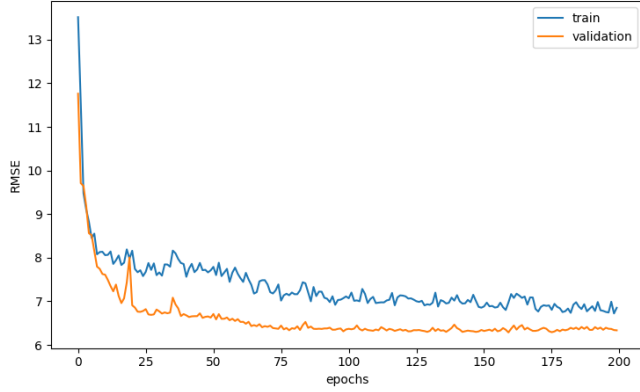


Training and validation history

Train and validation MAE history



Train and validation RMSE history



Train and validation MAPE history

